Pimpri-Chinchwad Education Trust's
Pimpri-Chinchwad College of Engineering
(An Autonomous Institute)
Affiliated to Savitribai Phule Pune University (SPPU)
ISO 21001:2018 Certified by TUV

Assignment No 4

Name: Soham Kulkarni
PRN: 124B1F056
Subject: Design and Analysis of Algorithms
Batch: A3
GitHub Link: https://github.com/SOHAM0007-CODER/DAA_ASSIGNMNET

1. Topic

Model a city road network as a weighted graph and route an emergency vehicle from its source to a selected hospital or destination using Dijkstra's algorithm. Use nonnegative travel costs and recompute the route after a road-weight update.

2. Course Objective

- Understand the computational complexity of algorithms.
- Select a suitable algorithm design strategy for a practical problem.

3. Course Outcome

- Find a minimum-cost path using greedy selection and edge relaxation.
- Recover the route, handle weight changes, and compare matrix and heap implementations.

Theory

Dijkstra's algorithm keeps a tentative distance for every vertex. The source starts at zero and other distances start at infinity. Repeatedly select the unvisited vertex with the smallest distance, finalize it, and relax its outgoing edges. Nonnegative edge weights make this greedy finalization safe.

The program uses an undirected adjacency matrix and long long costs. It completes the reachable-source search to print a correct distance table for every node. Each trace row shows the finalized node and successful relaxations. Road updates are checked and recomputed.

Working of the Algorithm

- 1. Read the road network and print its road table.
- 2. Select the nearest unvisited reachable node and relax its neighbors.
- 3. Print the trace, all-node distances, selected path and cumulative leg costs.
- 4. Accept a nonnegative road update and rerun the same source-destination request.

Complexity Analysis

Selecting the nearest vertex and scanning its matrix row take O(V) per iteration. Worst-case query time and matrix initialization are O(V squared).

Time Complexity

Best Case: Query may end when no reachable node remains; matrix setup is O(V squared).

Average Case: O(V squared).

Worst Case: O(V squared) per recomputation.

Space Complexity

O(V squared) for the adjacency matrix and O(V) for distance, parent and visited arrays. The supplied program has a fixed maximum of 1000 nodes.

Algorithm - Steps

- Step 1: Initialize distances, parents and visited flags.
- Step 2: Choose the smallest tentative distance and relax adjacent roads.
- Step 3: Follow parent links to print the route; recompute after a nonnegative update.

Pseudocode

```
dist[source]=0; other distances=infinity
repeat up to V times:
    u = nearest unvisited reachable node
    if no such u: stop
    mark u visited
    relax every unvisited neighbor and save parent
print all distances and recover destination route
```

Flow of Execution

START -> Read graph -> Select nearest vertex -> Relax edges -> Print path -> Update and rerun if needed -> END.

6. Example

The initial path is 0 -> 2 -> 1 -> 3 -> 4 with cost 10. The sample then changes road 2-3 to cost 1. The new path is 0 -> 2 -> 3 -> 4 with cost 5. Both runs include a distance table and route-leg breakdown.

Adjacency lists and a binary heap can improve sparse-network time to O((V+E) log V). BFS handles unweighted graphs; Bellman-Ford supports negative edges. This program uses a selected destination.

Conclusion

Dijkstra's algorithm finds a shortest route for nonnegative weights. Parent links recover the path, and recomputation handles updated costs. A heap-based version improves scalability.

Assignment 4 | SAMPLE INPUT (input.txt)

```
5 7
0 1 4
0 2 2
1 2 1
1 3 5
2 3 8
2 4 10
3 4 2
0 4
Y
2 3 1
N
```

Assignment 4 | FULL PROGRAM OUTPUT

```
ASSIGNMENT 4: EMERGENCY ROUTING - DIJKSTRA
Enter nodes (1-1000) and connections:
Enter undirected roads: from to nonnegativeCost (node IDs start at 0)

ROAD TABLE
From | To | Cost
0 | 1 | 4
0 | 2 | 2
1 | 2 | 1
1 | 3 | 5
2 | 3 | 8
2 | 4 | 10
3 | 4 | 2
Enter source and destination:

DIJKSTRA TRACE
Step | Finalized node | Minimum distance | Successful relaxations
1 | 0 | 0 | 1=4, 2=2
2 | 2 | 2 | 1=3, 3=10, 4=12
3 | 1 | 3 | 3=8
4 | 3 | 8 | 4=10
5 | 4 | 10 | None

DISTANCE TABLE FROM SOURCE 0
Node | Shortest cost | Parent
0 | 0 | -1
1 | 3 | 2
2 | 2 | 0
3 | 8 | 1
4 | 10 | 3

Optimal path: 0 -> 2 -> 1 -> 3 -> 4
Leg | From -> To | Road cost | Cumulative cost
1 | 0 -> 2 | 2 | 2
2 | 2 -> 1 | 1 | 3
3 | 1 -> 3 | 5 | 8
4 | 3 -> 4 | 2 | 10
Minimum cost: 10
Roads used: 4
Update a road? (Y/N):
Enter from to newCost:
Road updated: 2 <-> 3 cost 1. Recomputing.

DIJKSTRA TRACE
```

Assignment 4 | FULL PROGRAM OUTPUT (continued)

```
Step | Finalized node | Minimum distance | Successful relaxations
1 | 0 | 0 | 1=4, 2=2
2 | 2 | 2 | 1=3, 3=3, 4=12
3 | 1 | 3 | None
4 | 3 | 3 | 4=5
5 | 4 | 5 | None

DISTANCE TABLE FROM SOURCE 0
Node | Shortest cost | Parent
0 | 0 | -1
1 | 3 | 2
2 | 2 | 0
3 | 3 | 2
4 | 5 | 3

Optimal path: 0 -> 2 -> 3 -> 4
Leg | From -> To | Road cost | Cumulative cost
1 | 0 -> 2 | 2 | 2
2 | 2 -> 3 | 1 | 3
3 | 3 -> 4 | 2 | 5
Minimum cost: 5
Roads used: 3
Update another road? (Y/N):
Time per run: O(V^2) | Matrix storage: O(V^2)
```